Preenchimento de Triângulos por Scanline

1 Introdução

O objetivo do algoritmo é rasterizar (preencher) um triângulo 2D pintando pixels linha a linha (scanlines). A versão considerada ordena os vértices por coordenada vertical (y), computa coeficientes angulares (dx/dy) para as arestas e, para cada scanline válida, interpola os extremos horizontais e pinta os pixels entre eles.

2 O algoritmo

Sejam os vértices dados por $P_1 = (x_0, y_0), P_2 = (x_1, y_1), P_3 = (x_2, y_2)$.

1. Converter coordenadas para ponto flutuante: usar x_i, y_i em ponto flutuante para permitir interpolações.
2. Ordenar verticalmente os vértices de forma que $y_0 \leq y_1 \leq y_2$. Isso garante que há uma única "troca de aresta" na linha $y = y_1$.
3. Calcular coeficientes angulares (inclinacões) das arestas em termos de dx/dy:

$$m_{\text{far}} = \frac{x_2 - x_0}{y_2 - y_0 + \varepsilon}, \qquad m_{\text{up}} = \frac{x_1 - x_0}{y_1 - y_0 + \varepsilon}, \qquad m_{\text{low}} = \frac{x_2 - x_1}{y_2 - y_1 + \varepsilon},$$

   onde ε é um pequeno valor ou unidade 1 na implementação original para evitar divisão por zero quando $y_j = y_k$.
4. Inicializar os extremos horizontais da scanline superiormente: $x_f = x_0$ e $x_t = x_0 + m_{\text{up}}$ (caso $y_0 \neq y_1$; senão tratar topo plano).
5. Para cada linha inteira y de y_0 até y_2 (com clipping aos limites da imagem):
   (a) Se y está dentro da imagem, converter os extremos x para inteiros (clamp aos limites) e pintar pixels entre $\lfloor x_{\min} \rfloor$ e $\lfloor x_{\max} \rfloor$.
   (b) Atualizar x_f com m_{far}.
   (c) Atualizar x_t com m_{up} enquanto $y < y_1$, caso contrário com m_{low}.
6. Tratar limites da imagem (clamping) e casos degenerados explicitamente.

3 Explicação matemática

3.1 Ordenação dos vértices

Ordenar por y cria duas bandeiras monotônicas: uma aresta que vai de y_0 a y_2 (aresta “far”) e duas arestas que cobrem respectivamente as faixas superior (y_0 a y_1) e inferior (y_1 a y_2). Isso permite varredura única de y_0 a y_2 e mudança de controle no ponto médio y_1.

3.2 Cálculo dos coeficientes angulares

A aresta entre (x_a, y_a) e (x_b, y_b) tem equação paramétrica ao variar y:

$$x(y) = x_a + \frac{x_b - x_a}{y_b - y_a}\,(y - y_a).$$

Assim o coeficiente angular usado em atualização incremental por linha é

$$\Delta x = \frac{x_b - x_a}{y_b - y_a}.$$

No código apresentado a implementação usa $(y_b - y_a + 1)$ na divisão para evitar divisão por zero e alinhar atualizações discretas, porém isso altera ligeiramente a interpolação real e pode introduzir deslocamentos de meio-pixel.

3.3 Interpolação por scanline

Para uma scanline fixa y, os dois extremos são

$$x_{\text{left}} = x_{\text{left}}(y), \qquad x_{\text{right}} = x_{\text{right}}(y),$$

obtidos por integração incremental das Δx de cada aresta. Pintam-se os pixels inteiros entre $\lceil x_{\min} \rceil$ e $\lfloor x_{\max} \rfloor$.

4 Casos especiais

4.1 Topo plano ($y_0 = y_1$)

Se $y_0 = y_1$ temos um topo plano: as duas arestas superiores são horizontais. O preenchimento inicia-se em $y = y_0$ usando a aresta inferior (de y_0 a y_2) e a aresta que liga P_0 e P_2. Implementação: detectar $y_1 == y_0$ e inicializar os extremos usando as arestas relevantes; evitar divisão por zero no cálculo de Δx.

4.2 Base plana ($y_1 = y_2$)

Semelhante ao caso anterior: a parte inferior é plana. Preenche-se de y_0 até y_1 e trata-se a linha y_1 adequadamente.

4.3 Degenerações (colinearidade)

Se os três pontos são colineares verticalmente ou horizontalmente, a área é zero; o algoritmo deve detectar e ignorar ou desenhar a aresta conforme política desejada.

5 Clipping e limites de imagem

Antes de desenhar pixels:

- Truncar as faixas de y ao intervalo $[0, \text{height} - 1]$.
- Para cada scanline, clamp os extremos x ao intervalo $[0, \text{width} - 1]$.
- Evitar escrever fora do array de pixels para prevenir corrupção de memória.

6 Código exemplo

```
void FillTriangleSimple(point P1, point P2, point P3, int red, int green, int
    blue)
{
    // converter para double por precisao;
    double x0 = (double)P1.x, y0 = (double)P1.y;
    double x1 = (double)P2.x, y1 = (double)P2.y;
    double x2 = (double)P3.x, y2 = (double)P3.y;

    // ordenar por y: y0 <= y1 <= y2
    if (y1 < y0) { swap(&x0,&x1); swap(&y0,&y1); }
    if (y2 < y1) { swap(&x1,&x2); swap(&y1,&y2); }
    if (y1 < y0) { swap(&x0,&x1); swap(&y0,&y1); }

    // evitar divisao por zero; usar epsilon pequeno ou tratar casos planos
    double dy_far  = (y2 - y0);
    double dy_up   = (y1 - y0);
    double dy_low  = (y2 - y1);

    double dx_far  = (dy_far  != 0.0) ? (x2 - x0) / dy_far  : 0.0;
    double dx_up   = (dy_up   != 0.0) ? (x1 - x0) / dy_up   : 0.0;
    double dx_low  = (dy_low  != 0.0) ? (x2 - x1) / dy_low  : 0.0;

    double xf = x0;
    double xt = x0; // inicial
    int y_start = max( (int)ceil(y0), 0 );
    int y_end   = min( (int)floor(y2), height-1 );

    // caso topo plano (y0==y1): inicializa xt com x1
    if (dy_up == 0.0) xt = x1;
    else xt = x0 + dx_up;

    for (int y = y_start; y <= y_end; ++y)
    {
        if (y >= 0 && y < height)
        {
            int x_min = (int)ceil(min(xf, xt));
            int x_max = (int)floor(max(xf, xt));
            x_min = max(x_min, 0);
            x_max = min(x_max, width-1);
            for (int x = x_min; x <= x_max; ++x)
                putpixel(x, y, red, green, blue);
        }

        // atualizar incrementos de linha e coluna
        double local_y = (double)y + 1.0; // proxima linha
        xf += dx_far;
        if (local_y <= y1)
            xt += dx_up;
        else
            xt += dx_low;
    }
}
```